OTUS

Backend-разработка на Kotlin

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Распределенные системы и конкуретный доступ к базе данных

otus.ru

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
TG OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Команда курса

Окатов Сергей
автор программы и руководитель курса

- Кандидат физико-математических наук,
- 15 лет научной работы в университетах России, Европы, США;
- Руководитель отдела разработки БФТ.ЦР;
- Работа в стартапах и энтерпрайзе, BigData, Backend, Frontend.

Кирилл Крылов
преподаватель

- Kotlin/ML Разработчик в JetBrains;
- Веду курсы Android-разработки в Samsung;
- Full-stack разработчик мобильных приложений на Kotlin/TS;
- Руководитель курса Kotlin Basic.

Земенков Леонид
преподватель

- Kotlin разработчик в Ennabl, Inc.;
- 4 года пишу на Java;
- За 4 года успел поработать в Сбербанке, Yandex и AliExpress.

Непомнящий Евгений
преподаватель

- 15 лет программировал контроллеры на C++; и руководил отделом разработки
- 3 года пишу на Java;
- Последнее время пишу микросервисы на Java в M-Tex.

Таношкин Александр
преподаватель

- Инженер-программист в крупном классифайде недвижимости;
- Работал с Java, TS и другими языками;
- На Kotlin с 2019 г.

Порхун Анастасия
преподаватель

- 10 лет в индустрии
- Опыт работы в телеком- и аутсорскомпаниях, с гос. структурами;
- Проектирование и разработка ПО;
- Выстраивание процессов CI/CD, исследования в сфере ИБ.

Миних Владимир
преподаватель

- 1991 г. – моя первая программа (ZX Spectrum, Basic);
- 1998 г. – Delphi + первый Java опыт;
- 15+ лет – инженер АСУ ТП. Разработка ПО для ПЛК;
- с 2014 г. – Java разработчик, с 2018 г. – опыт с Kotlin.

Никольский Николай
преподаватель

- Java/Kotlin разработчик в ОТП банке.
- Работа с крупными государственными и банковскими проектами.
- Участник opensource

План полета

- Описание проблемы
- История распределенных систем
- Теория распределенных систем
- Реализация распределенных систем
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Знать особенности работы распределенных систем
2. Разрабатывать масштабируемые приложения
3. Строить бизнес-логику с учетом конкурентного доступа к БД

Смысл

Зачем вам это уметь

1. Распределенные системы позволяют разрабатывать HA (high availability, высокодоступные), HR (high resilience, высоконадежные) и масштабируемые приложения
2. Навык разработки распределенных приложений востребован на рынке. Практически все современные подходы именно распределенные.
3. Бизнес-логика в распределенных системах имеет свои особенности, необходимо их учитывать.

Общие сведения

Проблемы ПО 1990-х и 2000-х

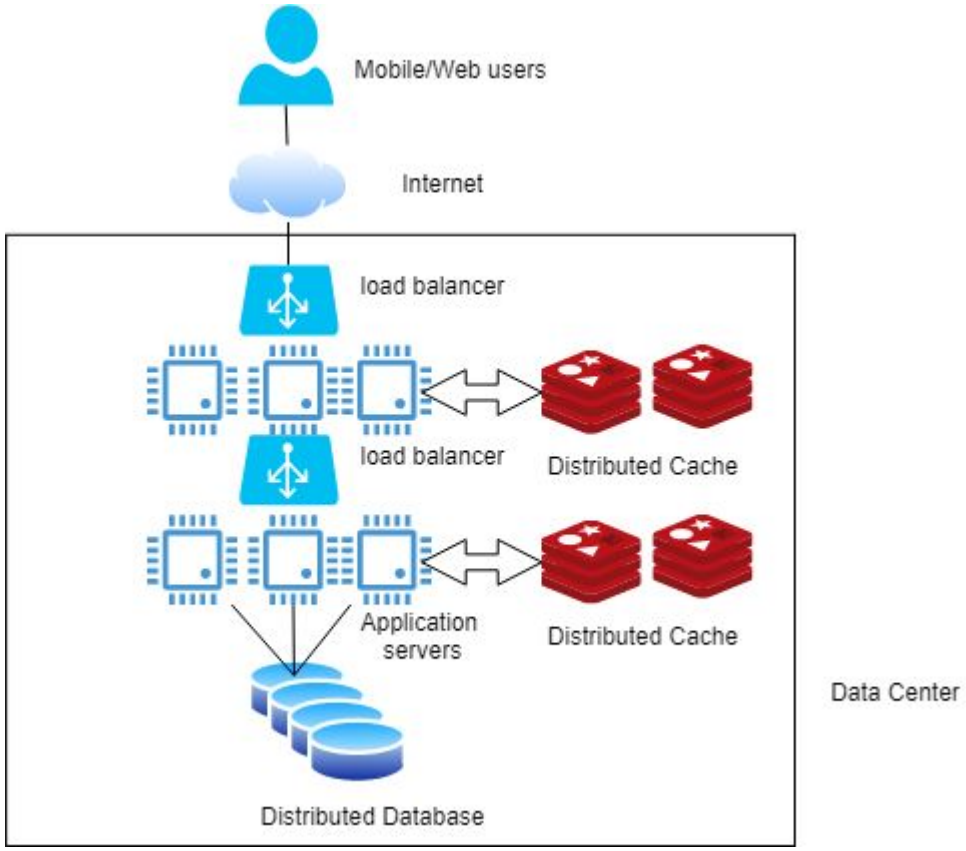
Особенности:

1. Монолитное приложение
2. Мультисессионность через процессы и потоки
3. SQL (ACID), конкуренция при работе с БД через транзакции
4. Эксперименты с MPI
5. Серьезные ограничения по производительности
    1. Блокировка транзакцией
    2. Ограничения базы данных
    3. Ограничения сервера приложений
6. Большое время простоя
    1. Сбой базы
    2. Сбой сервера приложений

Решения 2010-х

Особенности:

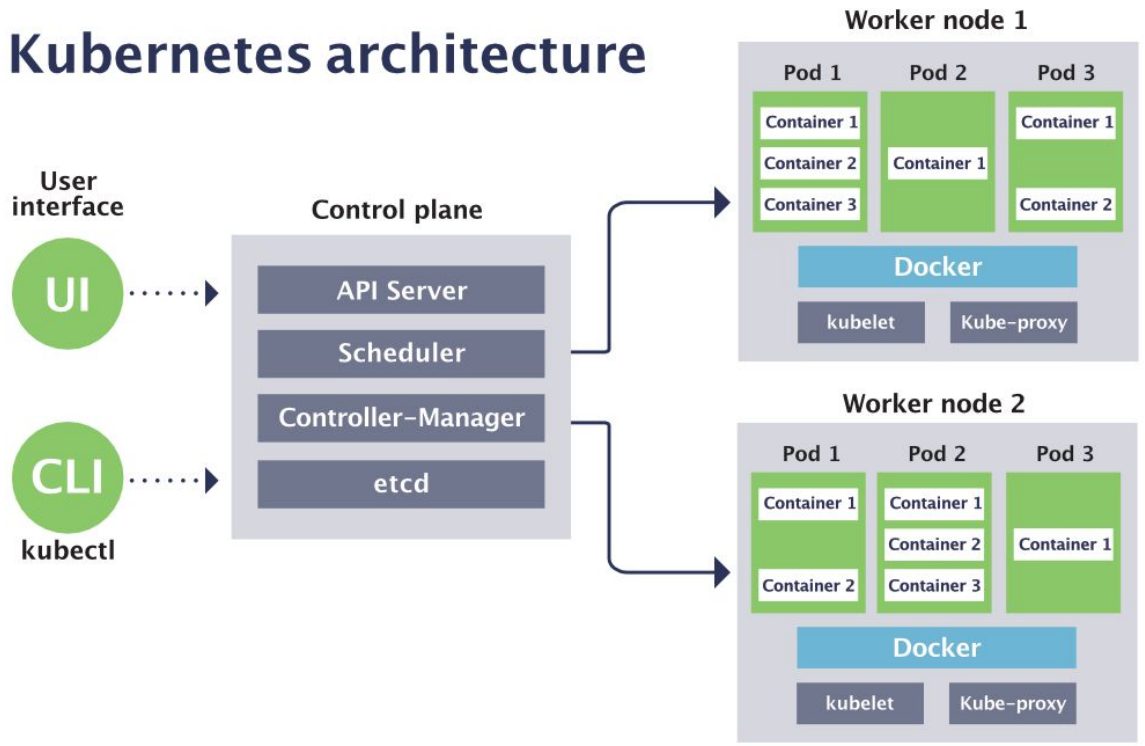
1. Микросервисная архитектура с собственной БД
    1. Снижение нагрузки на отдельный сервис
    2. Снижение нагрузки на инстанс БД
    3. Рост числа точек отказа
2. Масштабируемые приложения
    1. Защита от падений части узлов
    2. Гибкая реакция на нагрузку
3. NoSQL базы данных
    1. Масштабируемость хранения
    2. Защита от сбоев



Современные решения

Особенности:

1. Оркестрация микросервисов и их экземпляров (Kubernetes)
2. Постепенный перенос инфраструктуры в Kubernetes (БД, брокеры)
3. Облака и динамическое выделение ресурсов (serverless, горизонтальное и вертикальное масштабирование)
4. NewSQL решения

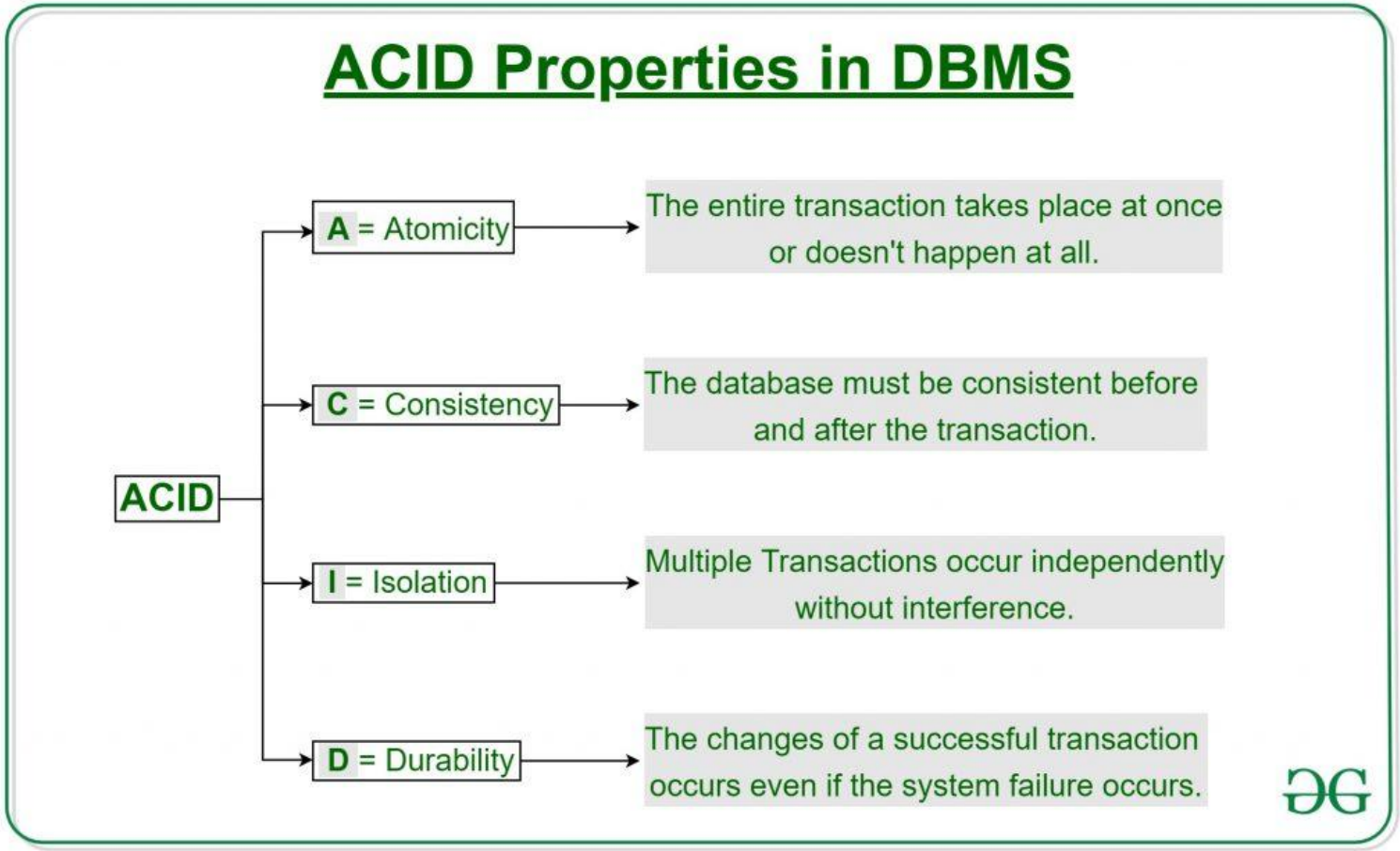


Элементы теории распределенных системы

Проблемы одноузельных систем

Особенности:

1. Не обеспечивают надежность
2. Ограниченная производительность
3. Высокая консистентность за счет ACID

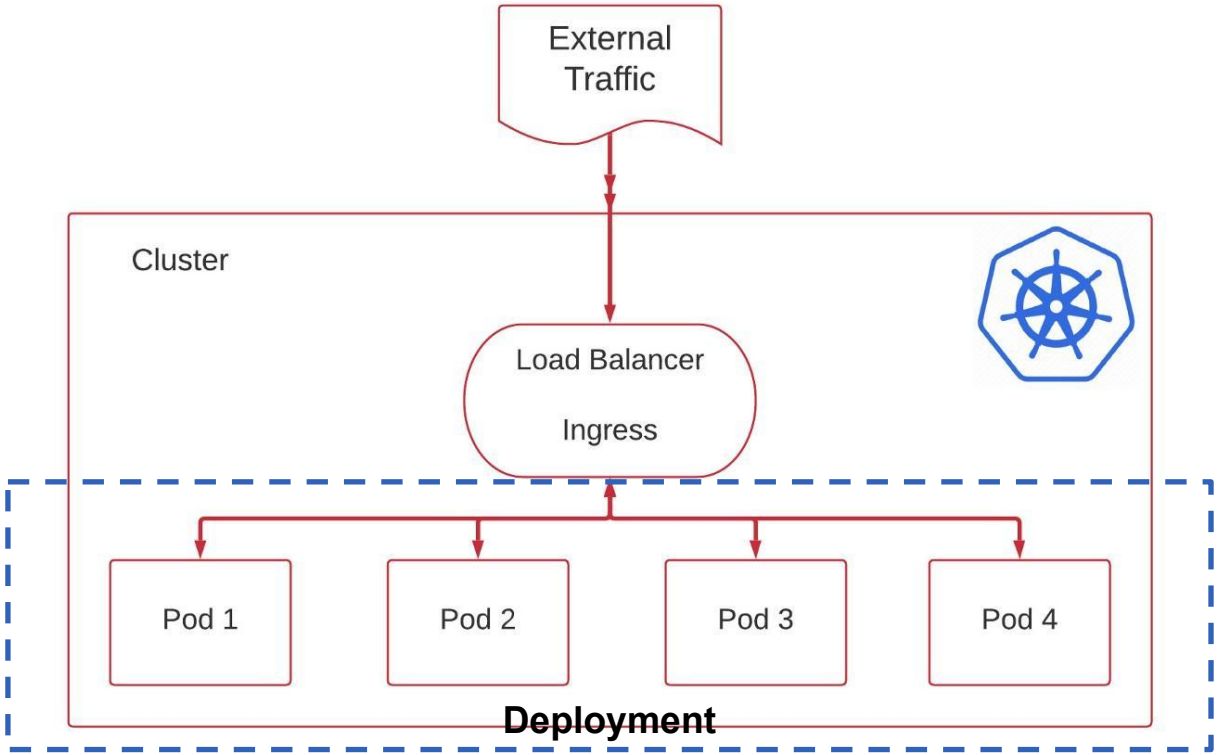


Работа распределенной системы

Пример нескольких балансировщиков: dig google.com

Особенности:

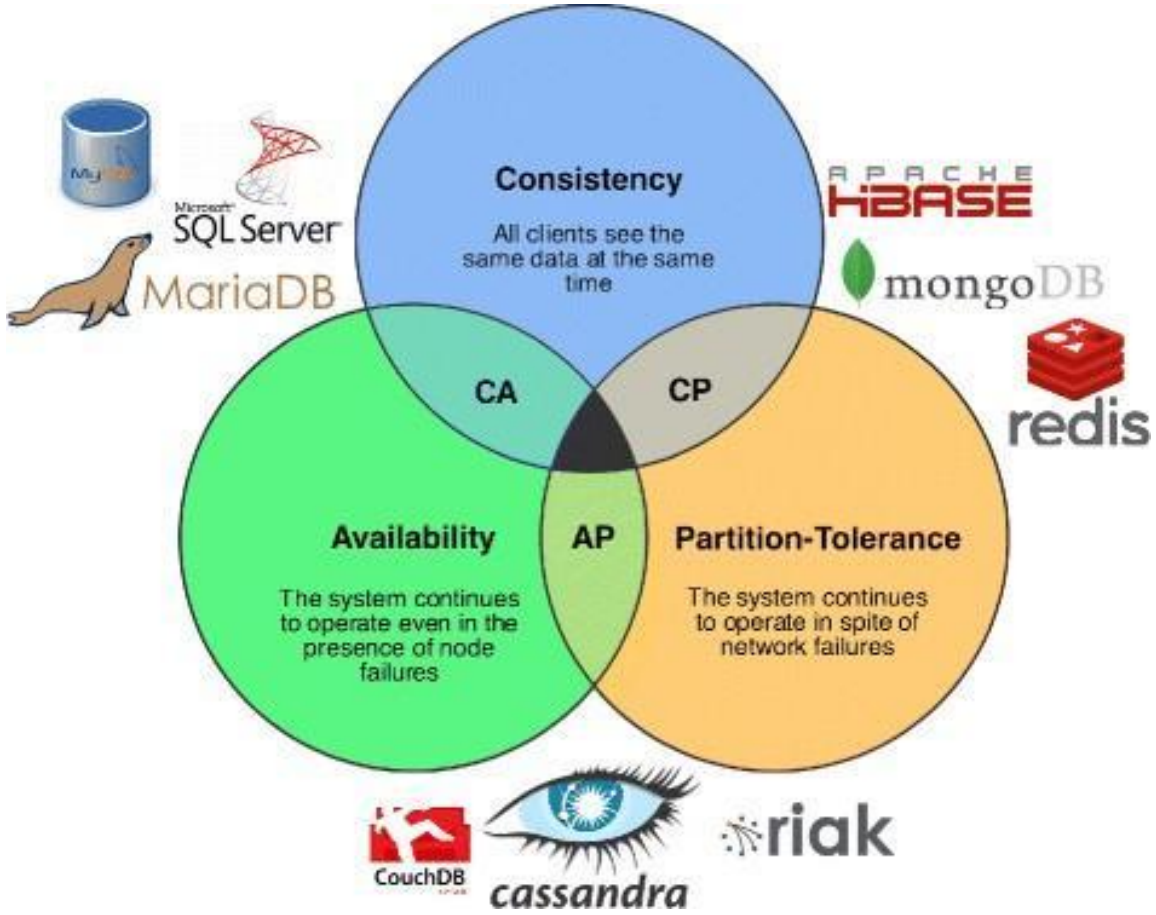
1. Каждый микросервис поднимается в нескольких экземплярах (подах) в рамках одного деплоймента (по-возможности, на разных узлах)
2. Каждый запрос обрабатывается на одном из подов (экземпляре приложения), выбранном квазислучайно
3. При падении пода, его работу должен взять на себя другой под



CAP-теорема

Особенности:

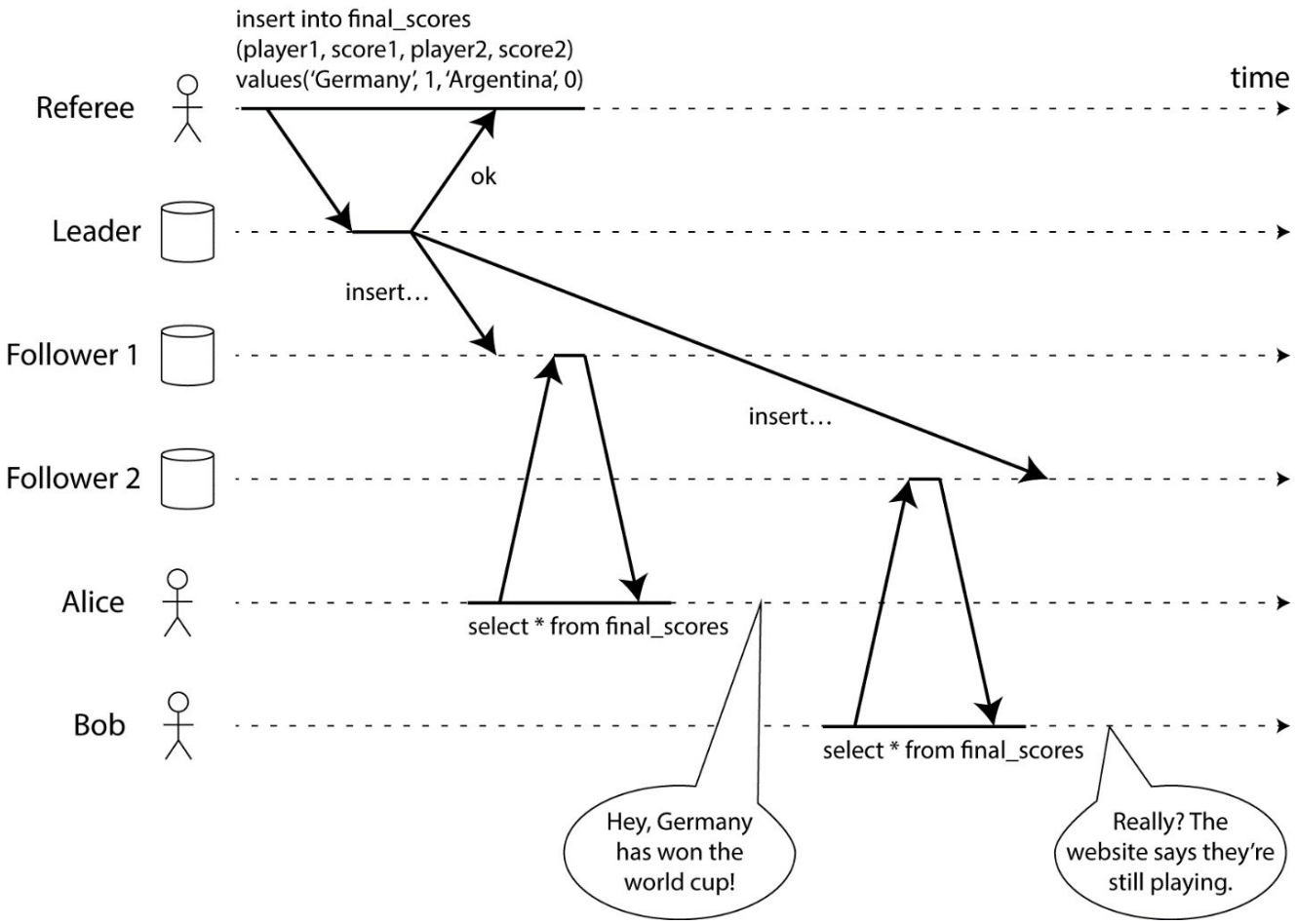
1. Описывает работу распределенных систем
2. Появилась в 1998-2000
3. Утверждает, что можно достичь только два из трех состояний системы: CA, CP, AP
4. Сыграла критическую роль в становлении NoSQL БД
5. До сих пор активно применяется, но уже есть более проработанные модели



Проблемы CAP-теоремы

Критика:

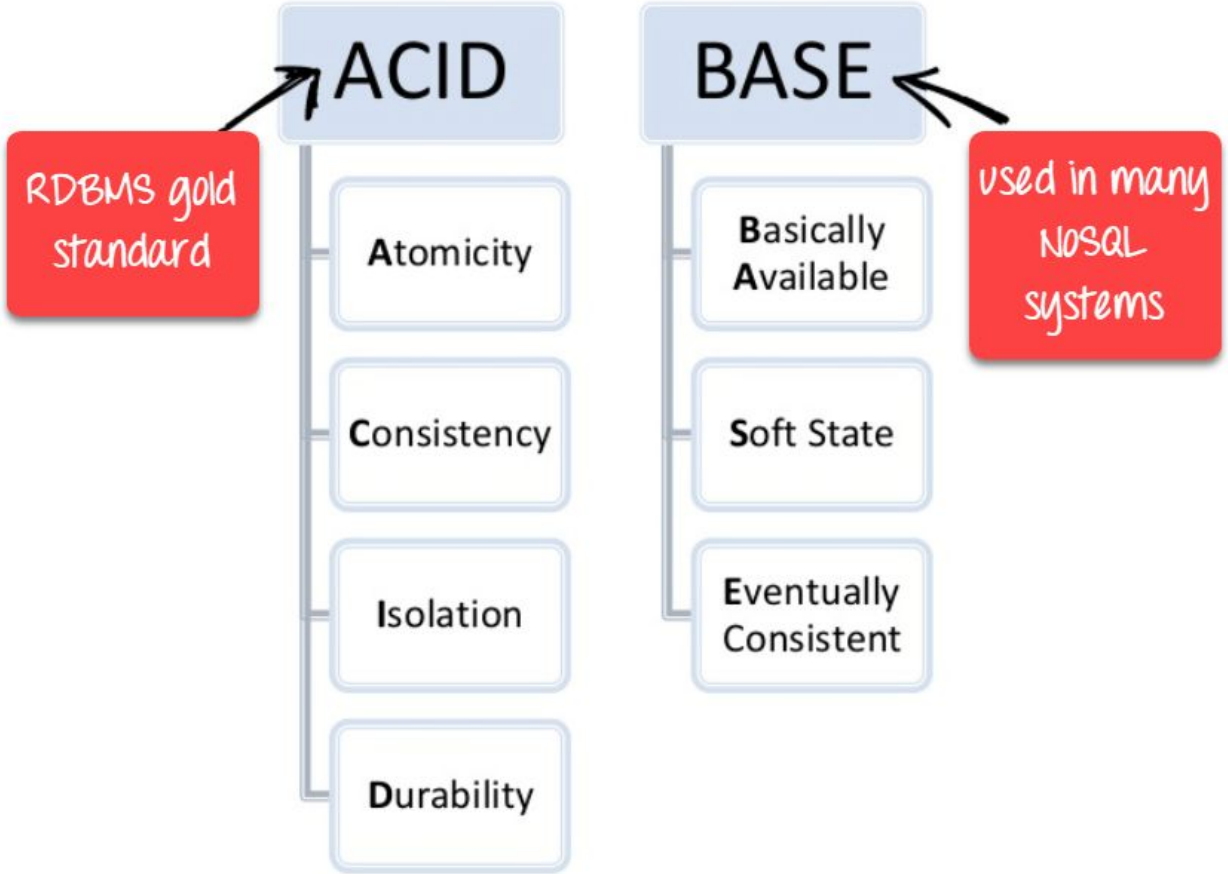
1. Описывает идеализированную абстрактную модель
2. Только бинарные состояния, не учитываются промежуточные состояния
3. Не учитывается латентность (временные задержки)
4. Не учитывается Eventual Consistency (конечную консистентность)



Расширения CAP-теоремы: BASE

Особенности:

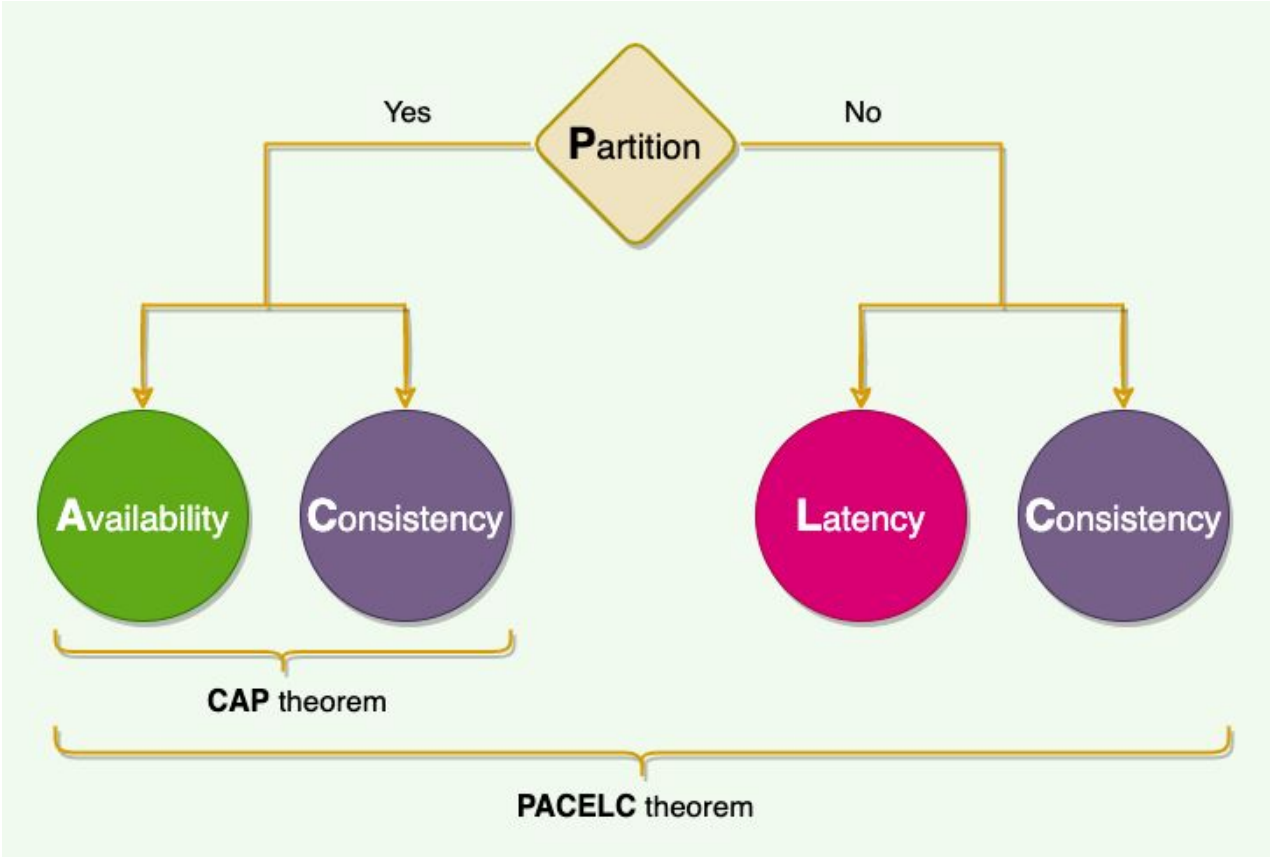
1. BASE (Basically-Available, Soft-state, Eventual consistency) принимает CAP-теорему, но пытается ее обойти через смягчение требований
2. Противопоставляется модели ACID
3. “Eventual consistency” – конечная консистентность (если не сейчас, но через какое-то время БД станет консистентной)
4. “Basic Availability” (базовая доступность) даже в случае сбоев за счет реплик
5. “Soft State” (мягкое состояние) – согласованность состояния – проблема разработчика, а не БД



Расширения CAP-теоремы: PACELC

Особенности:

1. Daniel J. Abadi from Yale University in 2010
2. Уточняет CAP (“новая версия”)
3. if **P** then **A** or **C**, else **L** or **C**: если система партицирована, выбираем доступность или согласованность, иначе латентность или согласованность
4. Архитектуры:
   1. PA/EL высокая доступность без согласованности
   2. PC/EC – высочайшая согласованность
   3. PA/EC
   4. PC/EL

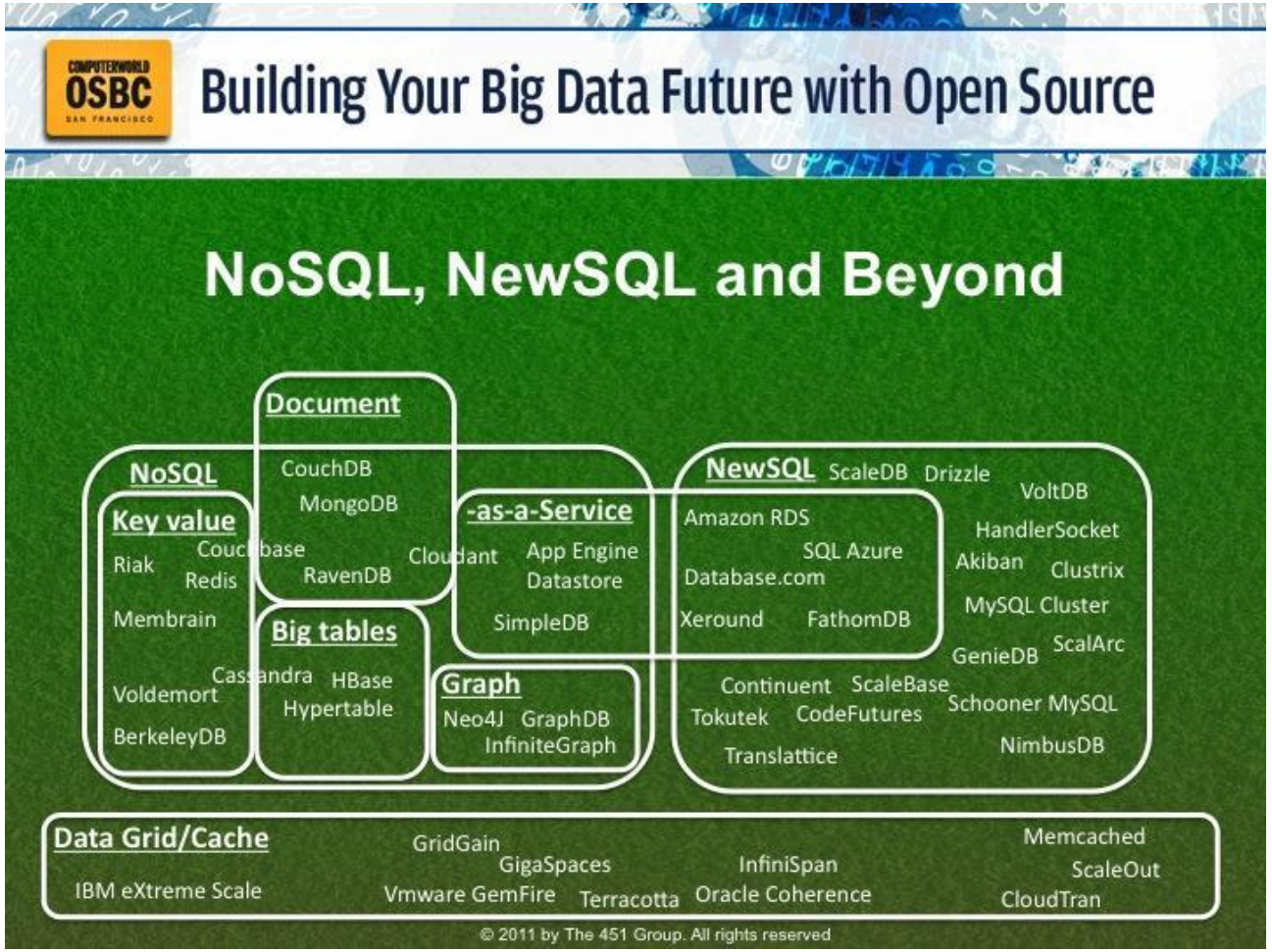


Распределенные БД

Современное состояние рынка БД

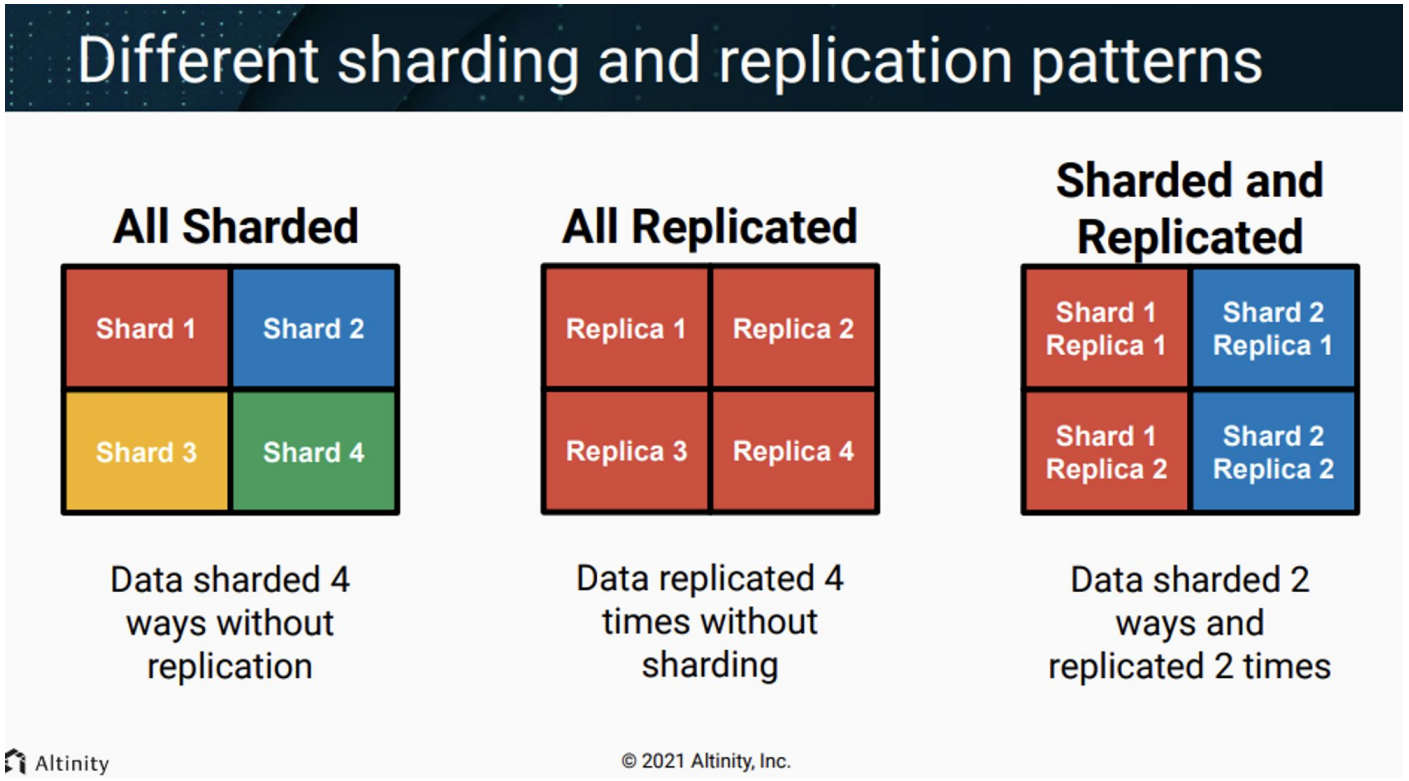
Виды:

1. SQL – MySQL, PostgreSQL, OracleDB, etc
2. NoSQL – Redis, HBase, Cassandra, AWS DynamoDB, etc
3. NewSQL – CockroachDB, VoltDB, YugabyteDB
    1. New Architectures
    2. SQL engines
    3. Transparent sharding
4. Graph DBs, TimeSeries DBs, Documentary DBs, etc
5. Multimodel DBs – ArcadeDB, ArangoDB, Virtuoso,OrientDB



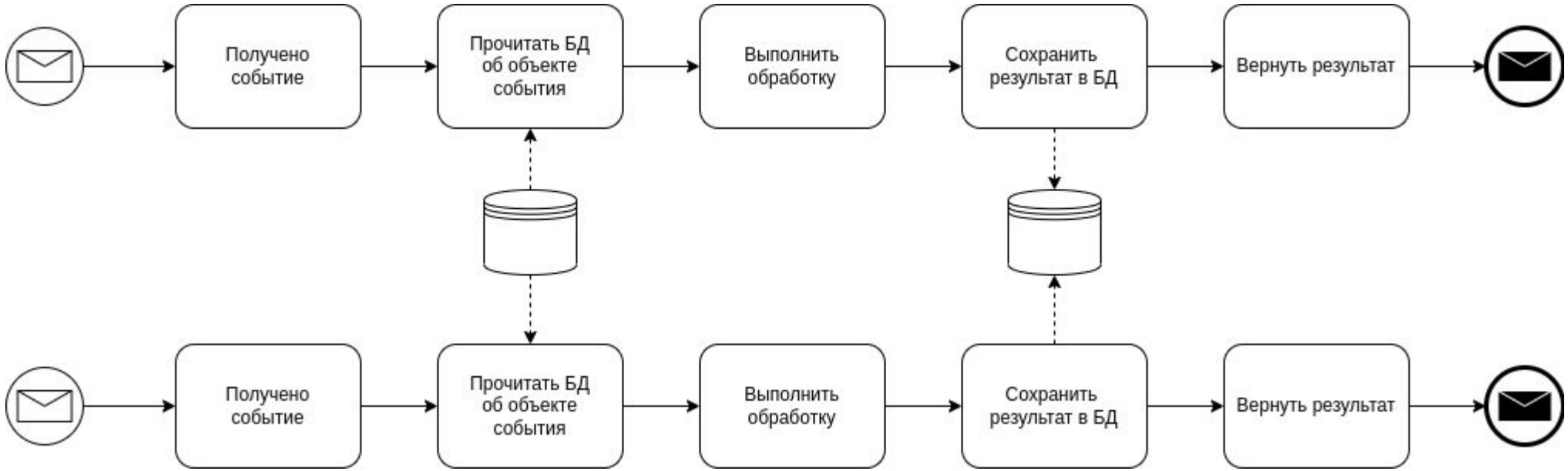
Репликация и шардирование

1. Репликация – дублирование одних и тех же данных на других узлах
2. Шардирование – распределение разных частей данных по разным узлам



Работа распределенной системы

Работа микросервиса в составе деплоймента



Особенности:

1. Каждый микросервис должен быть stateless (по возможности), т.е. хранить состояния только в БД
2. Два экземпляра микросервиса могут обрабатывать один объект в БД. Может приводить к конфликтам на уровне БД

Управление конфликтами между подами деплоймента

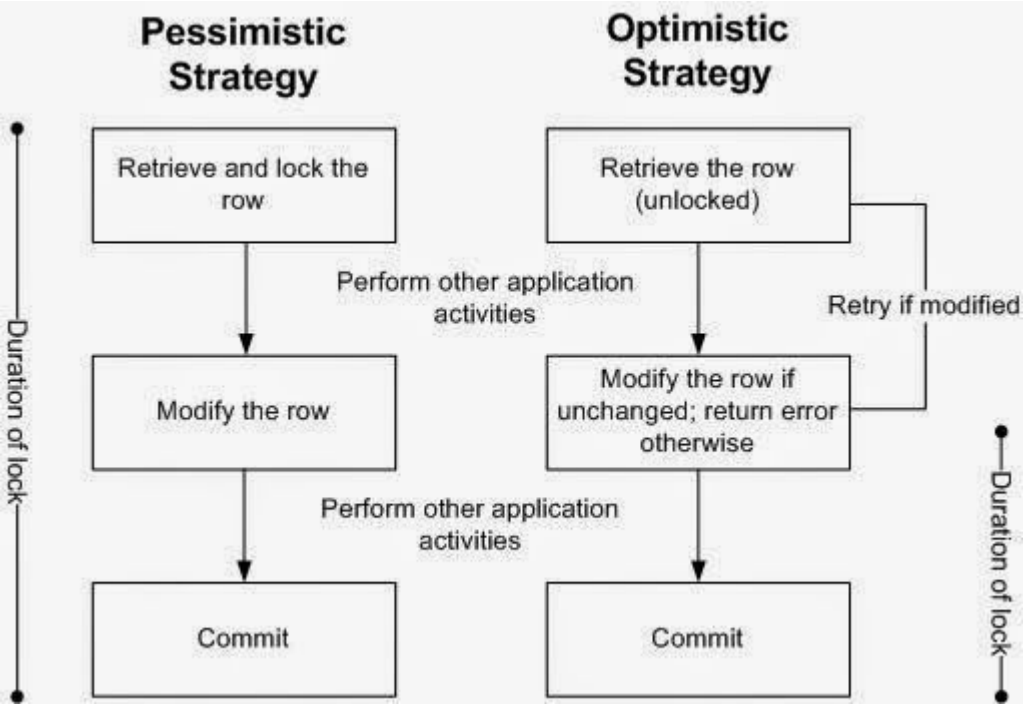
Управление конфликтами:

1. Отклоняем запись, возвращаем клиенту ошибку обновления записи с актуальной версией объекта (пользовательские формы)
2. Два экземпляра микросервиса могут обрабатывать один объект в БД. Может приводить к конфликтам на уровне БД (системные состояния)

Оптимистичная блокировка

Алгоритм:

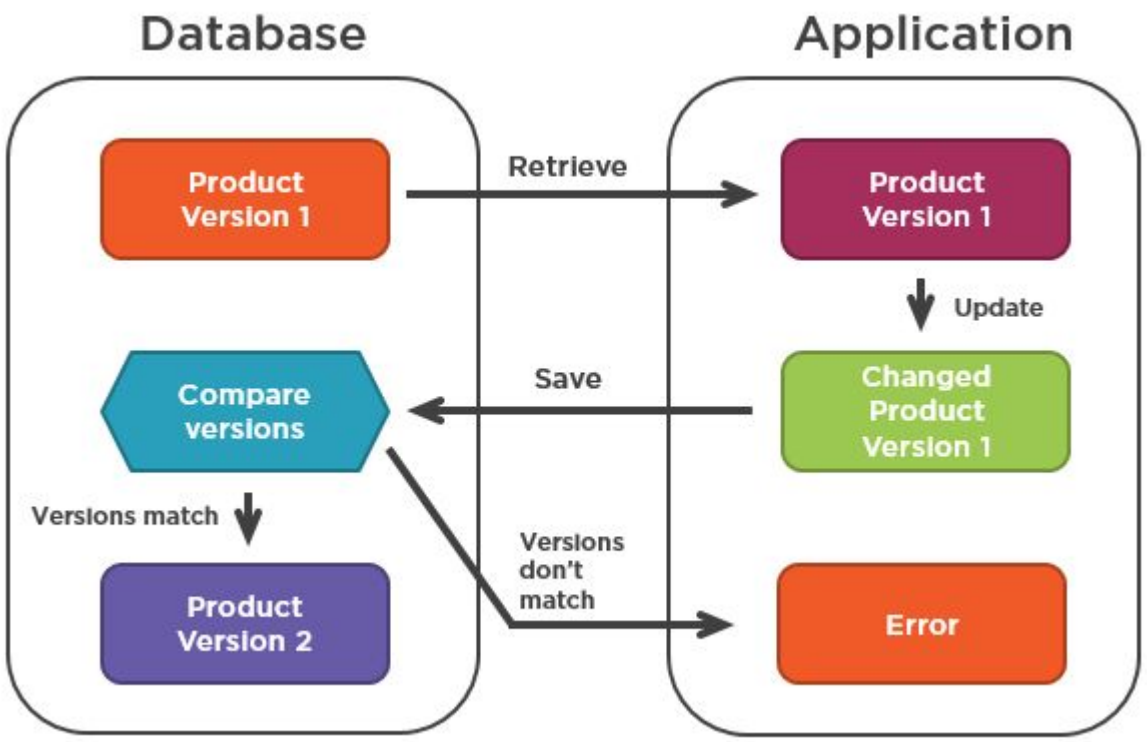
1. Без транзакции:
    1. Читаем объект из БД вместе с ключом блокировки
2. При сохранении включаем транзакцию
    1. Проверяем, что ключ блокировки не изменился в БД и сохраняем
    2. Если ключ блокировки изменился в БД, значит отказываем



Виды ключей блокировки

Виды:

1. Инкрементная версия
2. Случайная версия
3. Вычисляемый хэш по полям, требующим сохранить консистентность
4. Множественные версии
   1. Версия для применения результата 1
   2. Версия для применения результата 2
5. По предыдущей версии объекта: сравнение текущей версии в БД с прочитанной и требующейся записать



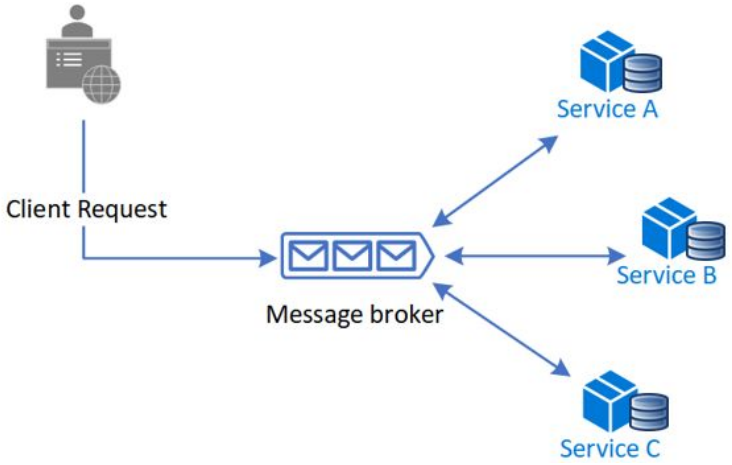
Синхронизация между микросервисами

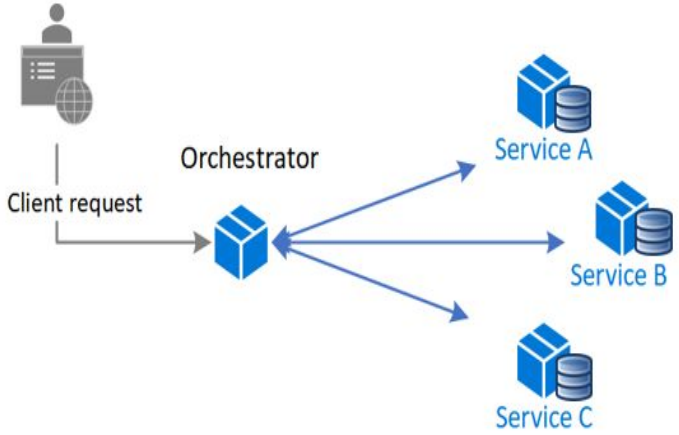
Пример:

при платеже счет отправителя и счет получателя должны остаться консистентными

Виды:

1. Распределенные двухфакторные транзакции: медленно, антипаттерн
2. Шаблон Saga – вместо отмены транзакции компенсационные транзакции
   1. Orchestrator
   2. Choreography



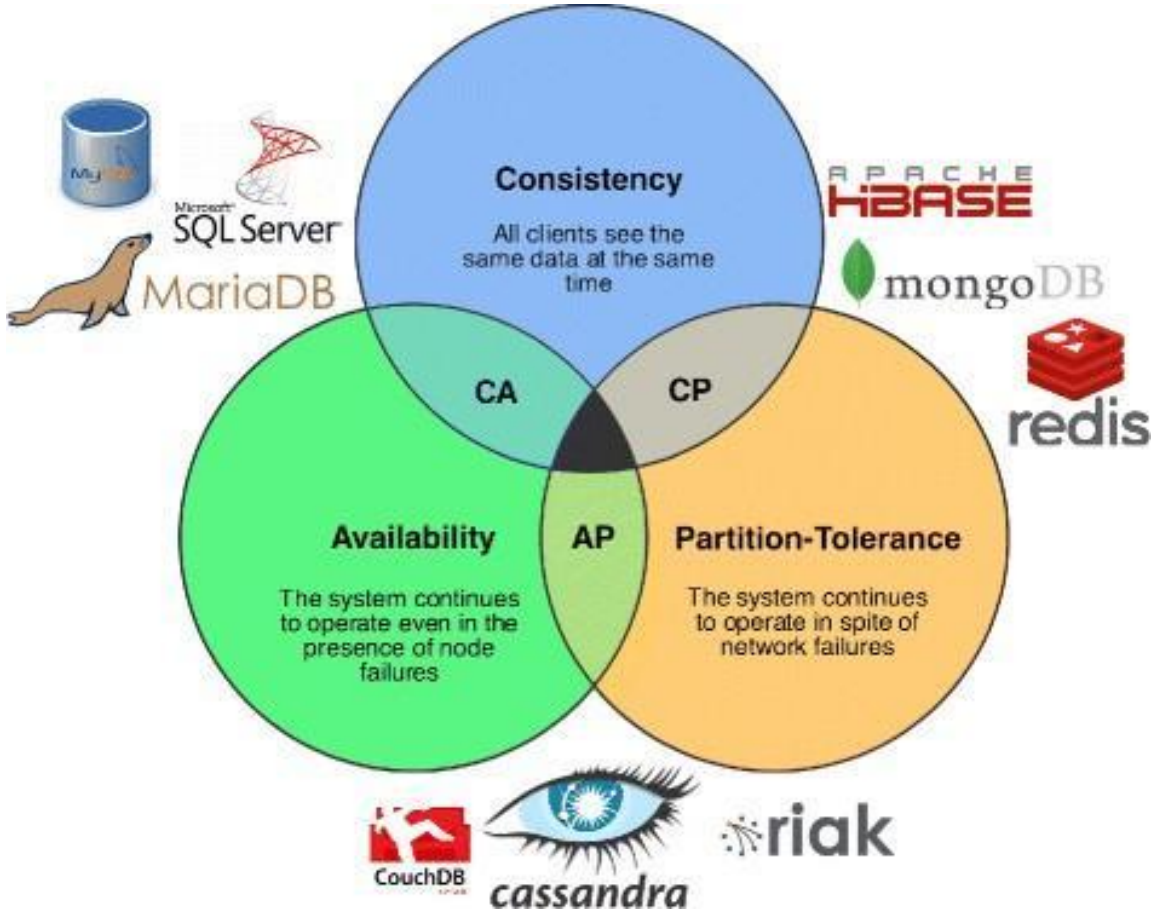


Ошибки работы с БД

Виды ошибок

Ошибки:

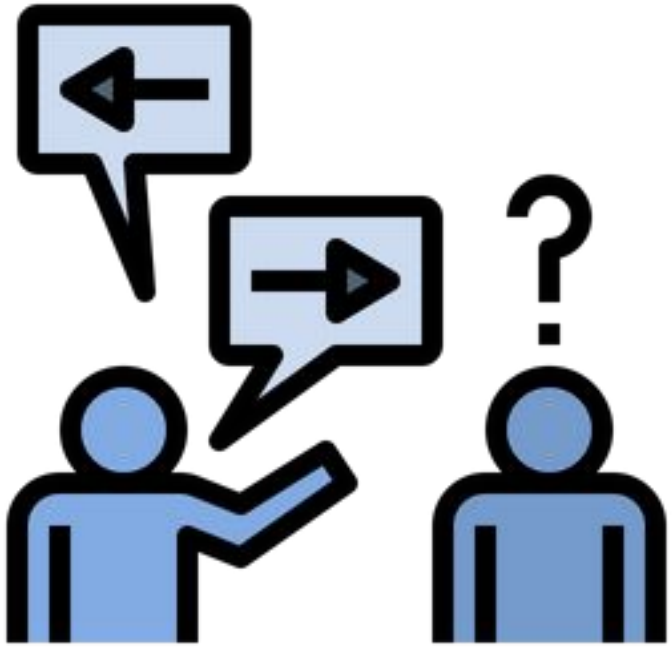
1. консистентности
2. доступности
3. партицирования (split brain)



Ошибки консистентности

Виды реакции:

1. Возвращаем ошибку клиенту – решение конфликтов вручную
2. Инкрементные обновления снижают вероятность и тяжесть последствий
3. Множественные попытки синхронизации (требуют идемпотентности)



Ошибки доступности

Виды реакции:

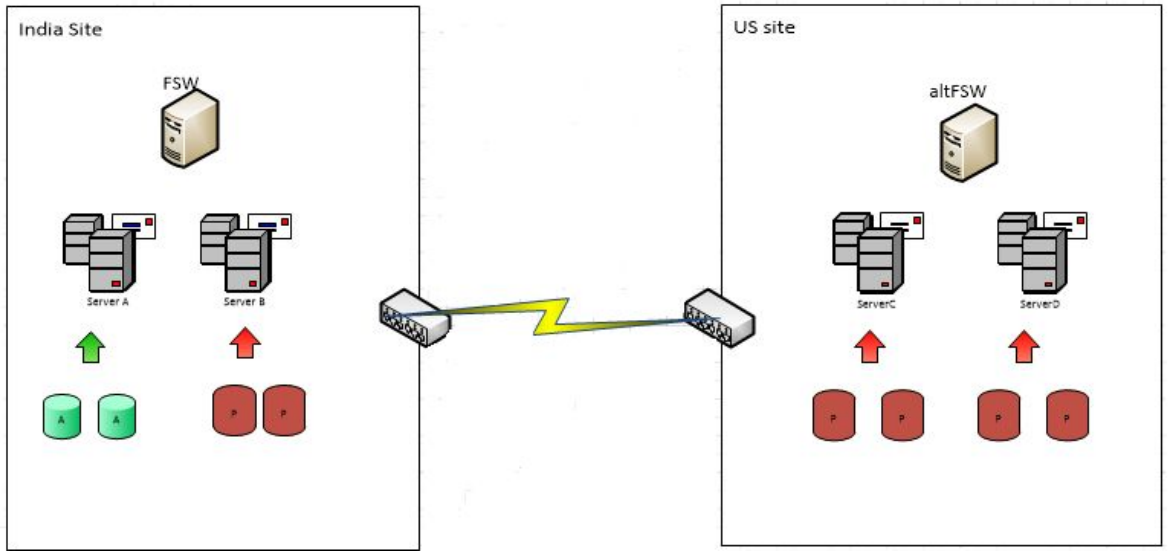
1. Повторяем запрос с увеличением периода ожидания

Требования: идемпотентность

Ошибки разделения сети

Виды реакции:

1. Обеспечиваем доступность, жертвуем согласованностью
2. Обеспечиваем согласованность, жертвуем доступностью
3. Ставим на ожидание (латентность)



LIVE

Рефлексия

Основные тезисы

1.	Современные решения строятся на базе Kubernetes в виде распределенных систем
2.	Модели для описания РС – CAP, BASE, PACELC
3.	Современные БД распределенные – NoSQL, NewSQL
4.	В бизнес-логику добавляется управление консистентностью – оптимистичные блокировки
5.	В логике обработка ошибок консистентности, доступности и разделения сети

Домашнее задание

1. Выполнить InMemory реализацию репозитария
2. Сделать автоматические тесты для репозитория
3. Создать персистентное хранилище для выпускного проекта
4. Интегрировать хранилище в выпускной проект

Срок: 17.10.2025

Рефлексия

По какому вопросу захотелось глубже изучить информацию?

Понимаете ли вы, как применять на практике то, что узнали на вебинаре. Если да, то как?

Цели вебинара

Проверка достижения целей

1. Знать особенности работы распределенных систем
2. Разрабатывать масштабируемые приложения
3. Строить бизнес-логику с учетом конкурентного доступа к БД

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/otuskotlin-user
- https://github.com/otuskotlin/202312-ok-marketplace
- CAP теорема: https://habr.com/ru/post/328792/
- PACELC теорема: https://habr.com/ru/company/gaz-is/blog/551986/
- BASE vs ACID: https://phoenixnap.com/kb/acid-vs-base

Следующий вебинар

07.10.2025

Модуль 7: Работа с базами данных

Репозиторий с PostgreSQL

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары